NEXUS: Cloud Backend Architecture

國泰金控 雲端技術暨專案實習生 — 面試前實作測驗 · 黃輝宏 Bertrand Leonard Widjaksono · Tamkang University CS

Repository: github.com/bertrand252/NEXUS · Live app: nexus-idx.vercel.app

中文摘要

NEXUS 是一個 AI 輔助的印尼股市（IDX）分析平台。後端以 Python FastAPI 撰寫，部署在 Railway；資料庫與登入驗證使用 Supabase（PostgreSQL + Auth，含 TOTP 雙因素驗證）；前端為 React，部署在 Vercel。後端內建 21 個背景排程任務，定時串接 yfinance、Invezgo、Groq LLM、Forex Factory 等第三方 API，計算股票評分，並透過 Telegram Bot 自動推播交易提醒。程式碼以 GitHub 管理，GitHub Actions 自動執行 122 個 pytest 測試與前端建置，push 到 main 後 Railway 與 Vercel 自動部署。

本專案的產品構想、功能規格與交易規則由我設計，程式實作大量使用 AI 輔助開發工具（Claude），使用方式與心得詳見第 7 節。

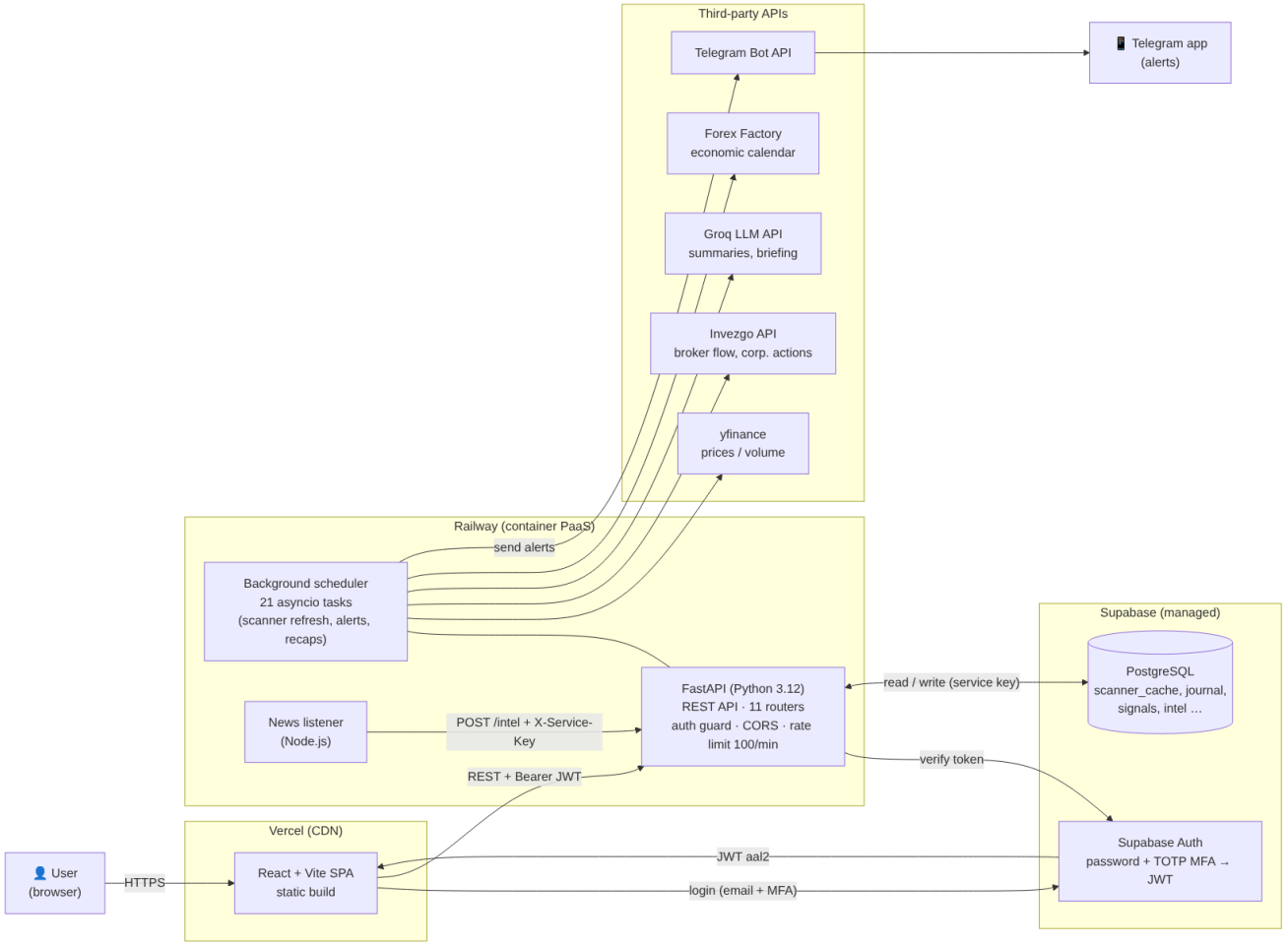
1. Purpose

Retail investors in Indonesia follow hundreds of stocks, broker-flow data, news channels and an economic calendar at the same time. NEXUS collects these sources automatically, scores every stock in the IDX universe (951 tickers) and sends a short, actionable alert to Telegram when a setup appears, so the user does not have to watch the market all day. It started as a capstone idea and is now a personal tool that I use every trading day.

2. Main features

Feature	What it does
Stock scanner	Scores 951 IDX tickers 0–100: Volume /25, Price momentum /25, Accumulation (broker flow) /30, Technical breakout /20. Results are cached in the `scanner_cache` table and refreshed automatically after market close.
Stock detail	Candlestick chart, support/resistance levels, XGBoost price-direction model, broker summary and foreign flow.
Telegram alerts	Automatic alerts for several trading styles (swing breakout, end-of-day "BSJP", intraday momentum), each with entry, target and stop-loss, plus large-transaction ("whale") alerts.
AI daily briefing	News from monitored channels is posted to `/intel`; the Groq LLM summarises it into a daily briefing that is grounded in the scanner's own signals.
Journal & analytics	Trading journal with calendar view, portfolio simulation and a history of every automatic call with its result (win rate, P&L).

3. Cloud architecture



Request flow (user opens the Scanner page)

1. The browser loads the React single-page app from Vercel's CDN.
2. The user logs in with email + password **and** a TOTP code; Supabase Auth returns a JWT with assurance level `aal2`.
3. The frontend calls `GET /scanner` on the FastAPI backend (Railway) with `Authorization: Bearer <JWT>`.
4. The backend's auth guard verifies the token with Supabase and rejects any token without MFA (`aal1`).
5. The router reads pre-computed scores from `scanner_cache` in PostgreSQL and returns JSON. Scores are pre-computed because fetching 951 tickers live on every page load is too slow.

Background flow (no user involved)

When FastAPI starts, its `lifespan` hook launches 21 `asyncio` background tasks in the same process (e.g. refresh the scanner at 16:00 WIB, check intraday momentum every 15 minutes during market hours, send the morning briefing at 08:45). Each task pulls data from the third-party APIs, writes results to PostgreSQL and sends alerts through the Telegram Bot API. A small Node.js listener forwards news messages to `POST /intel` using a machine-to-machine service key instead of a user login.

4. Cloud services and why I chose them

Service	Role	Why this service
Railway	Runs the FastAPI backend and the news listener as long-running containers.	The backend must run **24/7** because the scheduler sends alerts at fixed market times. Serverless functions that sleep between requests would miss those times. Railway runs a normal long-lived process, deploys automatically from GitHub, and stores secrets as environment variables.
Supabase	Managed PostgreSQL + authentication.	One service gives both a relational database and a ready-made auth system with TOTP MFA, so I did not have to build password storage or MFA myself. PostgreSQL fits the data well (journal, signals, cached scores are all tables).
Vercel	Hosts the React frontend.	The frontend is a static build, so a CDN is the cheapest and fastest option. A rewrite rule sends every path to `index.html` so client-side routing works.
GitHub Actions	Continuous integration.	Free for public repos and runs on every push and pull request.
Groq	LLM inference.	Fast and low-cost inference for the open model `openai/gpt-oss-120b`; the model name is one environment variable, so switching providers is a one-line change.

Trade-off: these PaaS products cost almost nothing at single-user scale and need very little operations work, but they give less control over networking and scaling than AWS/GCP.

The same design on AWS (for an enterprise setting)

NEXUS today	AWS equivalent	Note
Railway container	ECS Fargate service (behind an Application Load Balancer)	Long-running container, same Docker image
21 in-process asyncio tasks	EventBridge Scheduler → separate ECS tasks or Lambda	Lets the API scale to several instances without running each job twice
Supabase PostgreSQL	Amazon RDS for PostgreSQL	Private subnet, automated backups
Supabase Auth	Amazon Cognito (with MFA)	
Vercel	S3 + CloudFront	Static hosting on CDN
Railway env variables	AWS Secrets Manager	API keys and DB credentials
Railway logs	CloudWatch Logs + alarms	Alert when a scheduled job fails

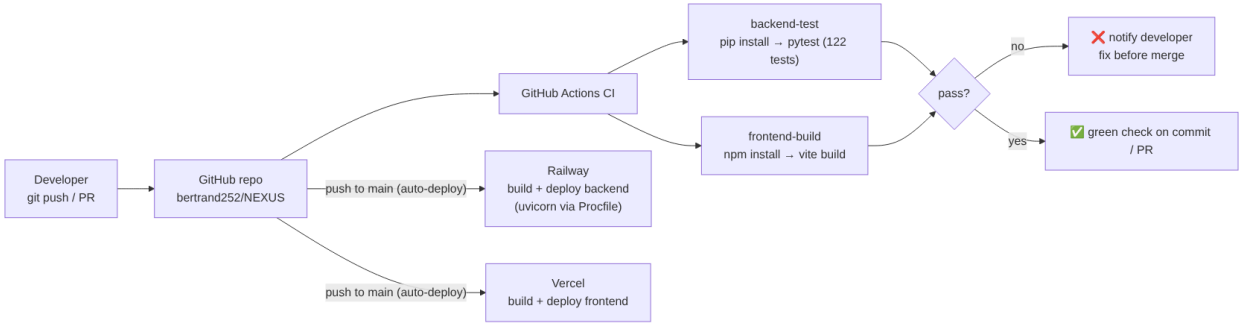
5. Security measures in the code

- **MFA enforced at the API**: `auth_guard.py` only accepts JWTs with `aal = "aal2"`, so a leaked password alone cannot access the API.
- **Service key for machines**: the news listener uses an `X-Service-Key` header, compared with `hmac.compare_digest` (constant-time, avoids timing attacks).
- **One auth gate**: authentication is attached once per router in `main.py`, not repeated per endpoint, so a new endpoint cannot be left unprotected by accident.
- **CORS** restricted to the frontend origin(s); **rate limiting** of 100 requests/minute per IP (slowapi).
- **Secrets** live only in environment variables; the repo contains `.env.example` files with placeholders.

6. Bonus items

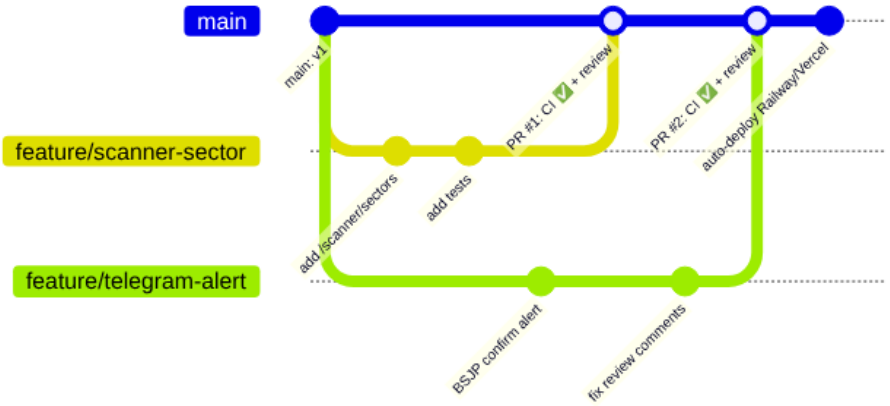
Item	Status	Details
1. Frontend	Done	React (Vite) + Tailwind + Chart.js + lightweight-charts, 10 pages, deployed on Vercel.
2. Third-party RESTful APIs	Done	yfinance, Invezgo (broker flow), Groq (LLM), Forex Factory (economic calendar), Telegram Bot API.
3. CI/CD	Done	GitHub Actions + automatic deploy; diagram below.
4. Git workflow for a team	Diagram	See below.
5. Infrastructure as Code	Not done	Infrastructure is configured through the Railway, Vercel and Supabase dashboards. Next step would be Terraform for the AWS version in section 4.
6. AI-assisted development	Done	See section 7.

CI/CD pipeline



CI runs two jobs on every push and pull request to `main`: `backend-test` (122 pytest unit tests on pure functions such as scoring, support/resistance levels and the auth guard, with dummy credentials so no real service is called) and `frontend-build`. Railway and Vercel deploy automatically from `main`. Current limitation: deployment does not wait for CI. In a team I would make CI a required check before merge (branch protection) so a failing build cannot reach production.

Git workflow for team collaboration



1. Each change starts on a feature branch from `main` (`feature/<name>`).
2. Small, focused commits; open a pull request when ready.
3. CI must pass and at least one teammate reviews the PR.
4. Merge to `main`; Railway and Vercel deploy automatically.

NEXUS itself is a solo project and I push to `main` directly. I use the feature-branch + pull-request flow above in my 4-person final project (Arca), where I am the frontend developer.

7. How I used AI tools

I used **Claude (Anthropic)** as my AI coding assistant for most of the implementation. My part was:

- **Product and rules**: I designed the features and the trading rules (what a BSJP or swing setup is, which data to use, when an alert should fire) from my own trading practice.
- **Directing and reviewing**: I described each feature, reviewed the result by running it, and asked for fixes when the behaviour was wrong. Many fixes came from real use, for example a BSJP order that was assumed filled although the real closing price had moved above my limit price.
- **Operating it**: I deployed it to Railway, Vercel and Supabase and use it every trading day.

What I learned: AI writes code quickly, but the quality depends on how clearly the requirement is described and how carefully the output is tested. Asking for automated tests and running them in CI caught mistakes that I would have missed by reading the code. I am still learning to read and explain every part of the codebase myself, and preparing this document was part of that.

8. Known limitations

- Single instance: the scheduler runs inside the API process, so running two instances would send duplicate alerts (fix: separate scheduler, as in the AWS design).
- Rate-limit counters are in memory and reset on redeploy (fix: Redis).
- Without an Invezgo subscription, the Accumulation score falls back to placeholder values; the other three scores always use real data.
- Code comments are written in Indonesian, my native language.